

EX NO: 8b	SIMULATION AND ANALYSIS OF QPSK MODULATION USING CONSTELLATION DIAGRAMS IN MATLAB
DATE:	

Objective

- To generate QPSK symbols from a binary data sequence using quadrature phase-shift keying principles.
- To plot and analyze the QPSK signal constellation in the In-Phase (I) and Quadrature (Q) plane.
- To investigate the effect of noise on constellation points and evaluate the resulting degradation in symbol detection performance.

Matlab code

```
/MATLAB Drive/experiment_8.m
1  clc;
2  clear;
3  close all;
4
5  N = 1000;
6  data = randi([0 1], N, 1);
7
8  if mod(N,2) ~= 0
9      data = [data; 0];
10     N = N + 1;
11 end
12
13 bit_pairs = reshape(data, 2, []);
14
15 qpsk_symbols = zeros(size(bit_pairs,1), 1);
16
17 for k = 1:size(bit_pairs,1)
18
19     if isequal(bit_pairs(k,:), [0 0])
20         qpsk_symbols(k) = (1 + 1j)/sqrt(2);
21
22     elseif isequal(bit_pairs(k,:), [0 1])
23         qpsk_symbols(k) = (-1 + 1j)/sqrt(2);
24
25     elseif isequal(bit_pairs(k,:), [1 1])
26         qpsk_symbols(k) = (-1 - 1j)/sqrt(2);
27
28     else
29         qpsk_symbols(k) = (1 - 1j)/sqrt(2);
30
31 end
32
33 figure;
34
35 scatter(real(qpsk_symbols), imag(qpsk_symbols), 30);
36
37 hold on;
38
39 xline(0, 'k', 'LineWidth', 1.5);
40 yline(0, 'k', 'LineWidth', 1.5);
41
42 grid on;
43 axis equal;
44
45 xlim([-1 1]);
46 ylim([-1 1]);
47
48 xlabel('In-Phase (I)');
49 ylabel('Quadrature (Q)');
50 title('Ideal QPSK Constellation');
51
52 SNR = 5;
53
54 rx_signal = awgn(qpsk_symbols, SNR, 'measured');
```

```

/MATLAB Drive/experiment_8.m
57 figure;
58 scatter(real(rx_signal), imag(rx_signal), 15);
59
60 hold on;
61
62 xline(0, 'k', 'LineWidth', 1.5);
63 yline(0, 'k', 'LineWidth', 1.5);
64
65 grid on;
66 axis equal;
67
68 xlabel('In-Phase (I)');
69 ylabel('Quadrature (Q)');
70
71 title(['Noisy QPSK Constellation (SNR = ', num2str(SNR), ' dB)']);
72
73 detected_bits = zeros(length(rx_signal) * 2, 1);
74
75
76 for k = 1:length(rx_signal)
77     I = real(rx_signal(k));
78     Q = imag(rx_signal(k));
79
80     if I > 0 && Q > 0
81         bits = [0 0];
82     elseif I < 0 && Q > 0
83         bits = [0 1];
84     elseif I < 0 && Q < 0
85         bits = [1 1];
86     else
87         bits = [1 0];
88     end
89
90     detected_bits(2*k-1:2*k) = bits;
91 end
92
93 num_errors = sum(data ~= detected_bits);
94
95 BER = num_errors / N;
96
97 fprintf('Number of Transmitted Bits : %d\n', N);
98 fprintf('Number of Bit Errors : %d\n', num_errors);
99 fprintf('Bit Error Rate (BER) : %f\n', BER);

```

Objective (i): Generate QPSK Symbols from a Binary Data Sequence Using Quadrature Phase-Shift Keying Principles

Procedure:

1. Initialize the MATLAB environment and define the number of bits.
2. Generate a random binary sequence consisting of 0s and 1s.
3. Group the binary sequence into pairs of bits.
4. Apply QPSK modulation by mapping each bit pair to a unique constellation point.
5. Generate the corresponding complex QPSK symbols.
6. Verify that four distinct symbol values are obtained.
7. Store the generated symbols for constellation analysis.

OUTPUT GRAPH

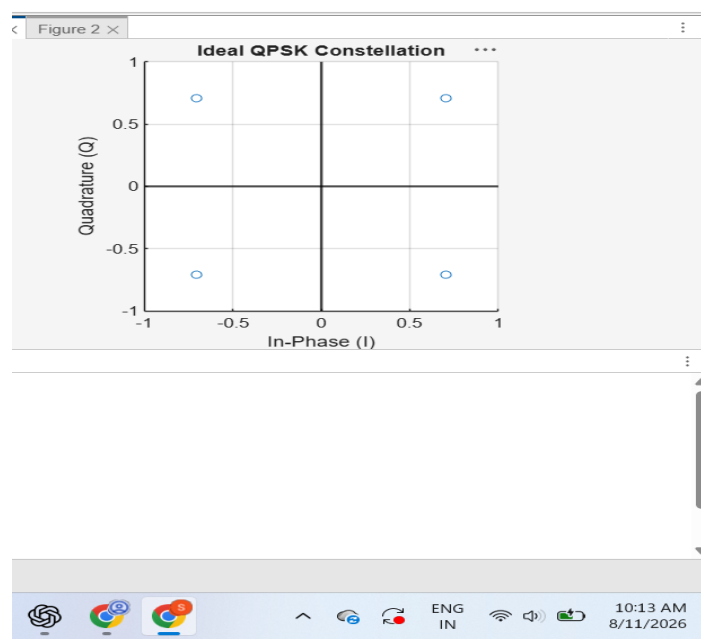


Fig. 1: Signal Constellation for QPSK Symbols

Result

QPSK symbols were successfully generated from the binary data sequence using quadrature phase-shift keying modulation.

Objective (ii): Plot and Analyze the QPSK Signal Constellation in the In-Phase (I) and Quadrature (Q) Plane

Procedure

1. Use the generated QPSK symbols for constellation plotting.
2. Extract the real component as the In-Phase (I) coordinate.
3. Extract the imaginary component as the Quadrature (Q) coordinate.
4. Plot the constellation points on the I–Q plane using a scatter plot.
5. Draw the horizontal and vertical axes through the origin.
6. Identify the four symbol locations corresponding to the four possible bit pairs.
7. Analyze the distribution of symbols in the four quadrants.

Output Graph

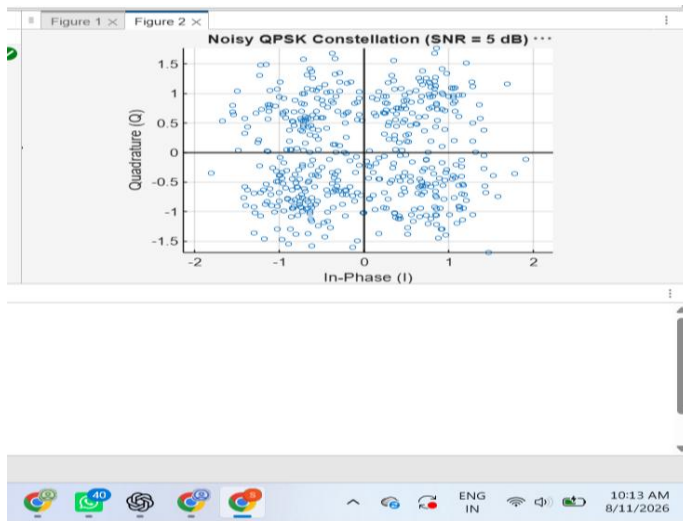


Fig. 2: Noisy QPSK Constellation

Result

The QPSK constellation was successfully plotted in the In-Phase (I) and Quadrature (Q) plane, showing four distinct symbol points corresponding to the bit pairs 00, 01, 11, and 10.